

CS406 Term Project Progress Report

Group 1

Umut Yunusoğlu
Selahattin Alp Özer
İbrahim Başar Demir
Aksel Dindisyan
Atakan Akpınar
Emir Gılbaz

January 15, 2026

1 Introduction

While decision trees are ideal for Explainable AI, computing Optimal Decision Trees (ODT) is an NP-hard problem which creates a massive computational workload. Traditional greedy methods like CART sidestep this complexity by compromising accuracy, but they fail to produce optimal results for continuous data.

The ConTree algorithm by Brita et al. addresses this accuracy gap using Dynamic Programming and Branch-and-Bound (BnB) to handle numeric data. However, despite introducing pruning techniques like neighborhood pruning (NB), interval shrinking (IS), and sub-interval pruning (SP), the algorithm remains bound by the limitations of serial execution. The search space grows exponentially with tree depth, creating a computational bottleneck that a single processor cannot efficiently handle. Crucially, the algorithm's heavy reliance on sorting continuous features and evaluating independent sub-intervals presents a distinct opportunity for parallel acceleration. The structured nature of these calculations makes them ideal candidates for a hybrid approach, exploiting the high-throughput capabilities of GPUs for data-parallel tasks and the task-parallelism of multi-core CPUs for managing the search tree.

2 Statement of Purpose

Our goal is to further improve on the speed and scalability of numeric data-based ODTs achieved by the authors of ConTree. While ConTree achieves its improvements over existing approaches through algorithmic innovations, our approach will instead focus on higher performance achievable through parallel execution of the ConTree algorithm. A hybrid parallelization approach utilizing both CPU and GPU parallelism will be employed. For CPU parallelism, OpenMP will be used, and for GPU parallelism, NVIDIA's CUDA will be used.

3 Bottlenecks In ConTree Implementation

In order to correctly identify the bottlenecks of ConTree we have performed a performance profiling. The profiling process was conducted on Intel(R) Core(TM) i7-14700HX (28CPU), 2.1GHz machine. As a profiling software the perf command and CLion profiler was used on a WSL environment.

Performance of Contree Algorithm on Avilla, Bank, Bean, Bidding, Eeg, Fault and Segment was examined. Nearly on all of the datasets a bottleneck in the specialised solver was detected.

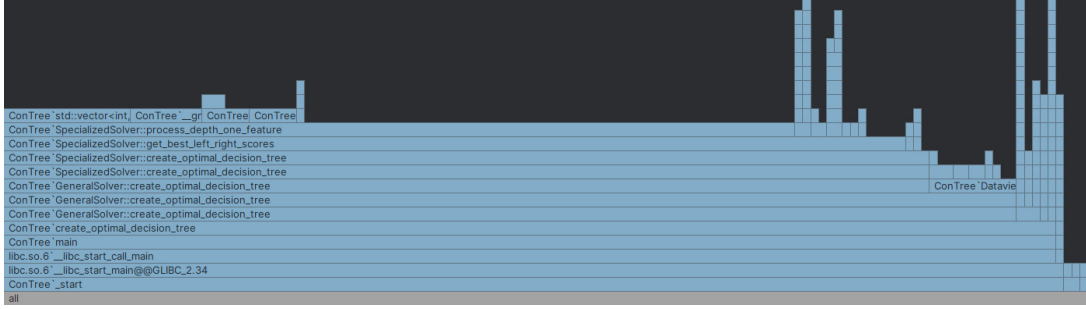


Figure 1: Profiling results of the ConTree implementation.

Figure 1 presents the flame graph from the profiling results bank dataset, demonstrating that the specialised solver dominates the runtime. Since the specialised solver is invoked at the completion of each depth-2 node, this computational overhead accumulates significantly. Upon analyzing the solver, we identified the source of the bottleneck: the solver calls a worker function for every feature. Within this worker function in order to find a depth one split, a nested loop iterates through all features again to execute the computationally intensive `process_depth_one_feature` routine. Because `process_depth_one_feature` scans the entire dataset, each call to the specialised solver results in F^2 full dataset iterations, where F is the number of features.

Additionally, effect of `process_depth_one_feature` routine can be seen in detailed assembly level profiling. 97% of cpu time is spent inside `get_best_left_right_scores`, and 75% of cpu time is spent inside `process_depth_one_feature`. This suggests that optimizing special solver will provide significant speedup.

```

1: get_best_left_right_scores(Dataview, SplitFeature, Threshold)
2: Initialize left_tree and right_tree structures;
3: // Loop over all features
4: for current_feature_index  $\leftarrow$  0 to feature_count do
5:   if current_feature_index == split_feature_index then
6:     Call process_depth_one_feature(... with sorted indices ...);
7:   else
8:     Call process_depth_one_feature(... with unsorted indices ...);
9:   end if
10:  Check if optimal score reached (pruning);
11: end for

12: process_depth_one_feature(Dataview, FeatureIndex) Reset label frequencies;
13: // Linear Data Scan
14: for i  $\leftarrow$  0 to dataset_size do
15:  Load data point;
16:  Update current_label_frequency (Prefix Sum);
17:  Calculate split score for Left/Right subtrees;
18:  Update best_split if score improves;
19: end for

```

Figure 2: ConTree Serial Specialized Solver Algorithm

Furthermore, we examined the general solver, which serves as the structural backbone of the algorithm. This component utilizes Dynamic Programming (DP) and pruning techniques to construct the optimal tree up to depth two. Our analysis reveals that the recursive procedure calls within the solver can be reformulated as independent parallel tasks, presenting a significant opportunity for performance optimization.

4 Parallel ConTree Implementation

4.1 CPU and GPU Hybrid Parallelization

As detailed in Section 3, the primary bottleneck of the specialised solver stems from the linear data scan required to identify the optimal depth-1 split point. To mitigate this latency, we restructured the algorithm to offload the loop calling **process_depth_one_feature** to a CUDA kernel. This approach allows us to exploit the massive parallelism of GPU architectures, effectively distributing the computational load of the linear scan.

4.1.1 Machine Resource Allocation

In order to maximize our cpu core utilization, we have implemented a simple algorithm to preassign thread numbers to omp parallel region. Below, you can see the code. Firstly, we check whether the dataset is bigger than 10000 instances.

If it is bigger than 10000, we open a loop where we try to find the lowest inner-threads value that is divisible with max thread number. After we find the suitable inner thread value, the (max_thread / inner_thread) value is assigned as the outer_thread value. This outer_threads and inner_threads combination is used in order to minimize inner-threads while using the maximum amount of available threads. Inner_threads (minimum

2) are minimized because the effectiveness of parallelizing intervals drop significantly when individual intervals are too small. Through `outer_threads` we maximize feature-level parallelization at depth 2 which creates independent sub-problems.

If there are less data points than 10000, then we just assign 1 to `inner_threads` and allocate all threads to the `outer_threads` if number of all threads are less than feature number. Otherwise, we assign the feature number as the outer thread number. We assign 1 to the inner threads because parallelizing small intervals between threads are inefficient.

```

1      int max_threads = omp_get_max_threads();
2
3      // Get feature number
4      const int feature_number = dataview.get_feature_number();
5
6      int inner_threads = 2;
7      int outer_threads = 1;
8
9      if(instance_num > 10000){
10         for(inner_threads; inner_threads < max_threads;
11             inner_threads++){
12             if(max_threads % inner_threads == 0){
13                 outer_threads = max_threads / inner_threads;
14                 break;
15             }
16         }
17     }
18     else{
19         inner_threads = 1;
20         outer_threads = std::min(max_threads, feature_number);
21     }

```

Figure 3: Dynamic thread allocation logic based on instance size.

4.1.2 CPU Parallelism for Specialised Solver

The specialised solver operates by iterating over features to identify potential depth-two splits, subsequently determining the optimal depth-one split for each candidate. We parallelised this outer feature iteration using OpenMP. Specifically, we employed a `parallel for` directive with a `schedule(static, 1)` configuration using `outer_threads` many threads. This strategy assigns feature processing tasks to CPU threads, each of these threads then spawn `inner_threads` many threads. The initial unsearched interval is divided into `inner_threads` many sub-intervals. This approach leads to some unnecessary computation as each sub-interval operates independently they are unable to benefit from pruning as effectively. However the benefits of parallelizing the interval search justifies this redundant work as it facilitates the concurrent execution of the CUDA kernels responsible for depth-one calculations. This setups allows `outer_threads` x `inner_threads` CUDA kernels to be run concurrently.

4.1.3 Data Organization At GPU

One of the fundamental challenges in the GPU implementation was organizing and transferring data efficiently to mitigate `cudaMemcpy` overheads. The specialised solver utilizes a `dataview` object to identify the subset of samples required for a split. However, the original ConTree architecture employed an Array of Structures (AoS) layout for both dataset and dataview implementations. Furthermore, the repeated copying of `dataview` objects for each specialised solver invocation proved computationally inefficient.

To mitigate the latency caused by redundant `cudaMemcpy` operations during each specialised solver invocation, we implemented a `gpu_context` class. This class restructures the host dataset into a flattened, contiguous memory layout arranged by feature (effectively a Structure of Arrays format, represented as $[f_{1,1}, f_{1,2}, \dots, f_{2,1}, \dots]$). Where the first subscript refers to the feature and the second refers to the datapoint, i.e., $f_{1,2}$ refers to the second datapoint's value for the first feature. This reorganization enables a single bulk transfer of the entire dataset to the GPU prior to the initialization of the tree construction, thereby eliminating the overhead of iterative data copying.

Additionally a mask object is used by the GPU kernel to keep track of datapoints belonging to various nodes within the tree structure. Each point has three possible values: 0,1,2 meaning not part of this node, falls under the left subtree of this node and falls under the right subtree of this node, respectively. This masking strategy avoids unnecessary data copying to all nodes.

4.1.4 GPU Parallelism for Specialised Solver

The special solver iteratively tries non-pruned split points for a node at depth two, before it can decide on the quality of a split it first has to know the optimal splits for the children of this depth two node. This is where the GPU kernel comes in, the kernel is responsible for finding the best split thresholds for both depth one children of the current node.

The CUDA kernel is launched as part of the `get_best_left_right_scores` function within the specialized solver. The kernel is launched with `number_of_features` many blocks, each responsible for finding the best split for that feature. Each block consists of 256 threads resulting in `outer_threads` x `inner_threads` x `number_of_features` x 256 GPU threads in total when kernels are running concurrently. The kernel can be broken down into four distinct steps: Local Histograms, Global Scan, Split Evaluation and Global Reduction.

Local Histograms: The first step is performed at the register level where each thread computes the class frequencies of its assigned data chunk as a histogram.

Global Scan: The second step ensures all threads have access to global class frequencies before split evaluation can begin. Each warp performs a warp-level reduction using shuffle down sync and write their totals to a shared memory array. A prefix scan is performed over shared memory followed by another synchronization.

Split Evaluation: The third step calculates the scores for possible splits based on the calculated class frequencies. Each thread is responsible for determining the best split for both the left and the right child given its chunk of the sorted feature array.

Global Reduction: In the final step a global reduction is performed and one thread from each block writes their block's best split thresholds and scores for both child nodes into an array. This array is then copied to the host.

Require: Feature Vector V , Labels Y , Masks M
Ensure: Optimal Split θ^* and Score S^*

```

1: procedure FINDBESTSPLIT( $V, Y, M$ )
2:   Parallel Step 1: Local Histograms (Map)
3:   for each data point  $i$  in thread's local chunk do
4:     if  $M[i]$  is active then
5:       Update private class counters based on  $Y[i]$ 
6:     end if
7:   end for
8:   Parallel Step 2: Global Alignment (Scan)
9:   Reduce private counters within warps
10:  Write warp totals to shared memory
11:  Perform block-wide prefix sum over warps
12:  ▷ Now, every thread knows the exact class distribution up to index  $i$ 
13:  Parallel Step 3: Split Evaluation
14:  for each split candidate  $i$  in thread's local chunk do
15:    if  $V[i] \neq V[i + 1]$  then                                     ▷ Valid boundary
16:       $L_{dist} \leftarrow$  Retrieve prefix sum at  $i$ 
17:       $R_{dist} \leftarrow TotalCounts - L_{dist}$ 
18:       $Score \leftarrow \text{CALCULATESCORE}(L_{dist}) + \text{CALCULATESCORE}(R_{dist})$ 
19:      Update thread-local best if  $Score > S_{local}$ 
20:    end if
21:  end for
22:  Parallel Step 4: Global Reduction
23:  atomicMax(GlobalBest, ThreadBest)
24: end procedure

```

Figure 4: Depth One Solver Kernel

4.1.5 Streams For Overlapping Memory Operations with Computation

Each kernel invocation necessitates transferring a bitset to device memory, followed by computing the mask and retrieving the results to host memory. Given the high frequency of kernel calls within the specialised solver, these data transfer latencies significantly dominate the total runtime. To mitigate this overhead and enable the overlapping of computation with memory operations, we implemented a stream manager. This module governs resource allocation using a FIFO (First-In, First-Out) policy: prior to a kernel launch, the host acquires an available stream from the head of the queue, and subsequently releases it to the tail upon task completion.

4.2 Implementation For CPU Only Machines

Additionally, we have implemented our approach for CPU only machines. In the CPU implementation data structures are same as the original implementation in paper, as there is no need to optimize data structures for transmission. There are two approaches for parallelization.

4.2.1 Tree Search Parallelization

As described in Section 4.1.2, the specialized solver parallelizes the evaluation of depth-two splits across features using an OpenMP `parallel for` directive with `schedule(static, 1)`. This enables the solver to process all features concurrently. In addition to this depth-two parallelization, we also introduce an additional parallelization directive for depth level three which parallelizes the feature search in general solver. In depth-three dynamic scheduling is used as each feature might take different amount of time.

4.2.2 Interval Parallelization

When the dataset is large, parallelizing only across features is insufficient. To further speed up the `get_best_left_right_scores` routine, we assign two threads to the specialized solver for each feature. We split the initial interval into two disjoint sub-intervals, and each thread searches for the best score within its assigned range. The threads share their current best scores, enabling better pruning in the other thread. Updates to the current best tree are performed within an OMP Critical section to ensure correctness.

5 Ideas for Further Improvement

5.1 Data Based Dynamic Thread Configuration

The current implementation partitions threads based solely on the machine’s maximum resources. This approach fails to consider the needs of different feature numbers and dataset sizes, a more effective implementation could be achieved by examining `number_of_features x number_of_datapoints`. When a dataset has 2-3 features while having a large number of datapoints, the current implementation allocates 16 threads as `outer_threads` while providing 2 threads as `inner_threads` on Nebula (32 CPU core machine). This results in inefficient use of nested parallelism.

5.2 Multiple GPU Scaling

The current implementation uses only a single GPU even though Nebula offers 4 GPUs. The current algorithm can be scaled up to multiple GPUs by dividing the available GPUs between OMP threads. This approach would likely result in a significant performance increase in cases where a single GPU is already being fully utilized.

5.3 MPI

The described multiple GPU scaling can be applied to multiple machines as well through MPI. In very large problems where a single machine’s resources would be overexerted, dividing the lower depths of the subtree ($depth \leq 2$) among MPI processes might make solving the problem feasible.

6 Results

Dataset	Depth	CPU Thread Count	Time (in seconds)	Misclassification Count
Avila	3	32	18.318	8664
Bean	3	32	32.099	1748
Raisin	4	7	487.518	74
Segment	4	18	467.275	101
Skin	3	32	14.0250	7761
Skin	4	32	1347.9220	2607

Table 1: Nebula Experiment Results

Dataset	Depth	CPU Thread Count	Time (in seconds)	Misclassification Count
Avila	3	60	3.558	8664
Bean	3	60	5.006	1748
Raisin	4	60	229.449	74
Segment	4	60	201.477	101

Table 2: Gandalf Experiment Results

As evidenced by the above results our implementation’s strength lies in large datasets due to the interval splitting nested parallelization approach. In small datasets only feature level parallelization is used leading to weaker results.

7 How to Build and Run

- For running the CPU implementation open the cpu folder and follow the below commands.
- For running the GPU implementation open gpu folder and follow the below commands.
- Run these commands:
 - `cd code`
 - `mkdir build`
 - `cd build`
 - `cmake ..`
 - `cmake --build .`
- For running the file type `./ConTree -file <file_address> -max-depth <depth> -time <time_limit>`

Note: Please reference section 6 when choosing the time_limit parameter during testing (default value is 600 seconds). Both implementations try to utilize all CPU cores of the machine as such if those resources are not available performance may be worse than reported in section 6.

8 References

Brița, Cătălin E., Jacobus G. M. van der Linden, and Emir Demirović. "Optimal Classification Trees for Continuous Feature Data Using Dynamic Programming with Branch-and-Bound." In Proceedings of AAAI-25 (2025).